

Intelligent Network Configuration Assistant (INA)

A Policy-Enforced Network Governance and Traffic Control Framework

Course: CS 331 Computer Networks

Group Number: T009 • **Project ID:** 5

Date: September 11, 2026

GitHub Repository:

<https://github.com/vedantb7/CS331-T009-IntelligentNetworkAssistant.git>

Abstract

Modern network operations require automated, policy-governed, and verifiable mechanisms for packet filtering and traffic shaping. Manual execution of kernel networking utilities (`nftables`, `tc`) is error-prone, lacks standardized policy enforcement, and risks host misconfiguration. This report presents the **Intelligent Network Configuration Assistant (INA)**, an automated network governance framework evaluated within an isolated containerized bridge network as an experimental testbed environment.

INA combines natural-language intent interpretation, an independent policy engine, dynamic container lookup, and kernel-level enforcement primitives. Packet filtering is enforced at Layer 2 via `nftables` bridge set evaluation, while bi-directional bandwidth limits are enforced using Token Bucket Filter (`tc tbf`) queueing disciplines and Intermediate Functional Block (`ifb0`) ingress redirection. System state convergence is actively verified through an automated empirical monitoring layer running connectivity and throughput benchmarks. The complete system was evaluated across extensive automated functional and integration test suites, demonstrating reliable policy enforcement and controlled traffic management within target thresholds.

Keywords

Containerized Testbed, Linux Bridge Networking, Layer-2 Packet Filtering, `nftables`, Traffic Control (`tc` `tbfb`), IFB Ingress Redirection, Policy Enforcement, Empirical Network Verification.

Contents

Keywords	2
1 Introduction	4
1.1 Network Configuration and Management	4
1.2 Linux Networking Primitives Used in INA	4
1.3 Containerized Testbed Environment	4
1.4 Network Configuration Operations	4
1.5 System Architecture and Workflow	5
1.6 Modular Layer Architecture	6
2 Our Setup	6
2.1 Network Topology and Architecture	6
3 Implementation Details	7
3.1 Network Controller and Privileged Scope Trade-Offs	7
3.2 Dynamic Container Discovery	7
3.3 Network Configuration Tools	7
3.3.1 Block Client Operation (<code>block_client</code>)	7
3.3.2 Unblock Client Operation (<code>unblock_client</code>)	8
3.3.3 Limit Bandwidth Operation (<code>limit_bandwidth</code>)	8
3.4 Policy Engine Governance	8
3.5 Protocol-Driven Tool Execution	8
4 Network Configuration and Traffic Control	8
4.1 Layer-2 Packet Filtering Mechanics	8
4.2 Bi-Directional Traffic Shaping and IFB Redirection	8
4.3 Network Packet Paths and Baseline Throughput	8
4.4 Network State Transition Summary	9
5 Network Validation and Testing Setup	9
5.1 Validation Architecture	9
5.2 Overcoming Host Ping Routing Bypass	9
5.3 Empirical Throughput Verification	9
5.4 Empirical System Evidence Excerpts	10
5.5 Automated Test Suite Matrix	10
5.6 Handled Edge Cases	10
6 Results	10
6.1 Verification of Setup	10
6.2 Block / Unblock Verification Results	11
6.3 Dual-Direction Bandwidth Shaping Results	11
6.4 Network Behavior Analysis	11
7 Debugging, Root Cause Analysis, and Technical Challenges	11
7.1 Cross-Module Integration and Boundary Adapters	11
7.2 Policy Governance and Data Validation	11
7.3 Natural Language Intent Parsing and Resilience	12
7.4 Kernel Enforcement and Technical Breakthroughs	12

8	Security Considerations and Threat Model	12
9	Design Limitations and Future Improvements	14
9.1	Current System Limitations	14
9.2	Future Technical Proposals	14
10	Conclusion	14

1 Introduction

1.1 Network Configuration and Management

- **Configuration Management:** Network infrastructure management demands operational control over packet filtering, bandwidth allocation, node isolation, and routing policies.
- **Manual Complexity:** Administrative tasks traditionally rely on syntax-dense command-line tools (`nftables`, `tc`). Manual execution is prone to typos, rule precedence ordering bugs, and security misconfigurations.
- **Policy Enforcement Deficit:** Standard Linux utilities execute commands with root privileges without validating whether an operation violates organizational access control policies or targets critical infrastructure nodes.
- **Missing State Verification:** Standard configuration tooling applies rules without probing the network data plane to verify empirical state convergence post-execution.

1.2 Linux Networking Primitives Used in INA

- **Network Namespaces (`netns`) & Virtual Interfaces:** Provides isolated network stacks per workload node, linked to the bridge interface via virtual network interface pairs.
- **Linux Bridge & Netfilter (`br_netfilter`):** Software Layer-2 bridge connecting container interfaces. Loading `br_netfilter` enables `nftables` to intercept bridged frames at Layer 2.
- **Layer-2 `nftables` Filtering:** INA attaches an IP lookup set to the bridge family forward chain, dropping unauthorized frames before host Layer-3 routing takes place.
- **Traffic Control (`tc`) & IFB Redirection:** Egress rate limiting uses Token Bucket Filter (`tbfb`) queueing disciplines. Ingress shaping is achieved by redirecting incoming packets to an Intermediate Functional Block (`ifb0`) pseudo-device.

1.3 Containerized Testbed Environment

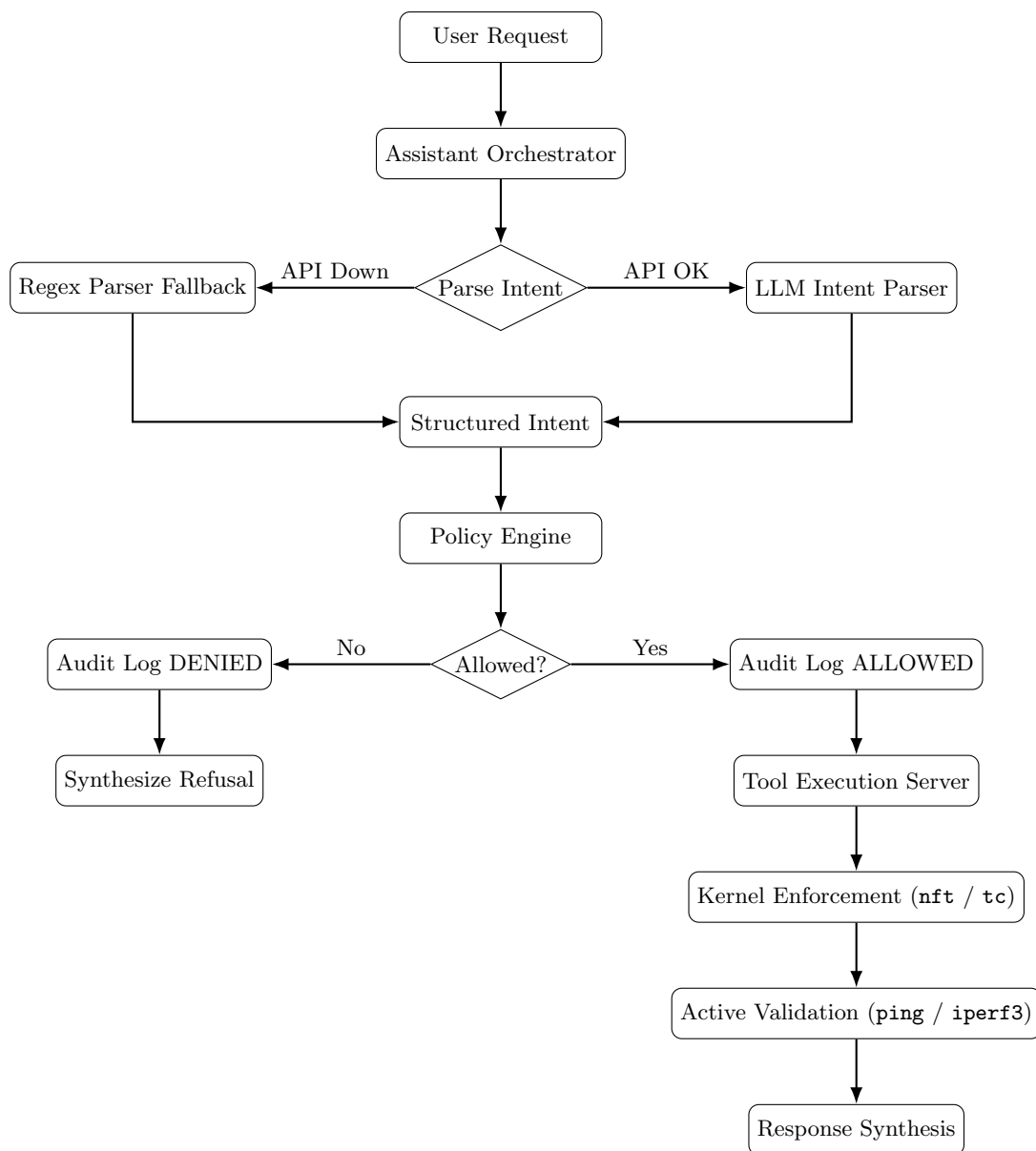
- **Docker Bridge Medium:** An isolated software bridge interface on the host system serves as the experimental testbed environment, assigning containers IP addresses within a private virtual bridge network.
- **Interface Topology:** Each workload container possesses a dedicated interface inside its isolated network namespace, bound to a corresponding virtual interface on the host bridge.
- **Capability Isolation:** Host networking and administrative privileges (`CAP_NET_ADMIN`, `CAP_SYS_ADMIN`) are strictly confined to a single network controller container, preserving unprivileged operation for workload nodes.

1.4 Network Configuration Operations

- **Client Isolation (`block_client`):** Adds the target IPv4 address to an `nftables` bridge set, dropping frames at Layer 2 before they reach host routing hooks.
- **Access Restoration (`unblock_client`):** Removes the target IP from the blocked set and triggers complete traffic control queueing discipline and virtual interface teardown.
- **Bi-Directional Throttling (`limit_bandwidth`):** Applies Token Bucket Filter (`tbfb`) queueing disciplines for egress shaping and redirects ingress traffic to an Intermediate Functional Block pseudo-device for ingress rate limiting.

1.5 System Architecture and Workflow

The INA pipeline translates high-level user requests into kernel-level network operations. Crucially, the LLM is used strictly for intent interpretation, while independent policy rules and controlled tools govern the resulting network operations. LLM output is treated as untrusted input and cannot directly invoke kernel commands.



1.6 Modular Layer Architecture

System components are partitioned into distinct architectural layers to prevent direct, unvalidated command execution.

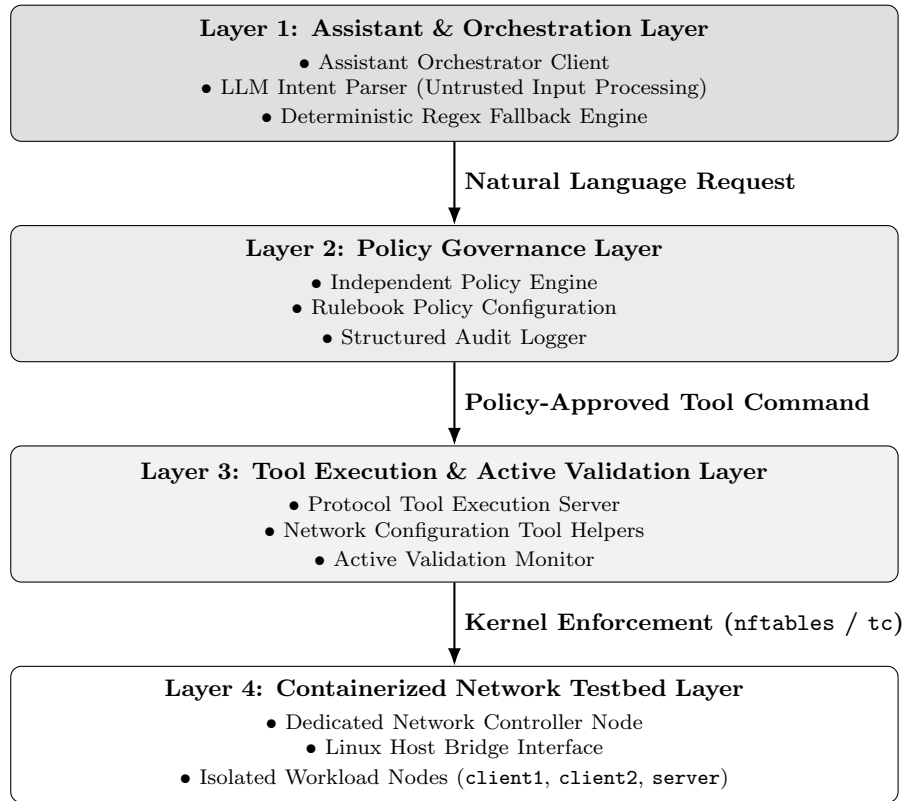


Figure 1: INA Modular Architecture Layer Breakdown

2 Our Setup

2.1 Network Topology and Architecture

The experimental network emulates a segmented LAN created using Docker containers operating on a private virtual bridge subnet.

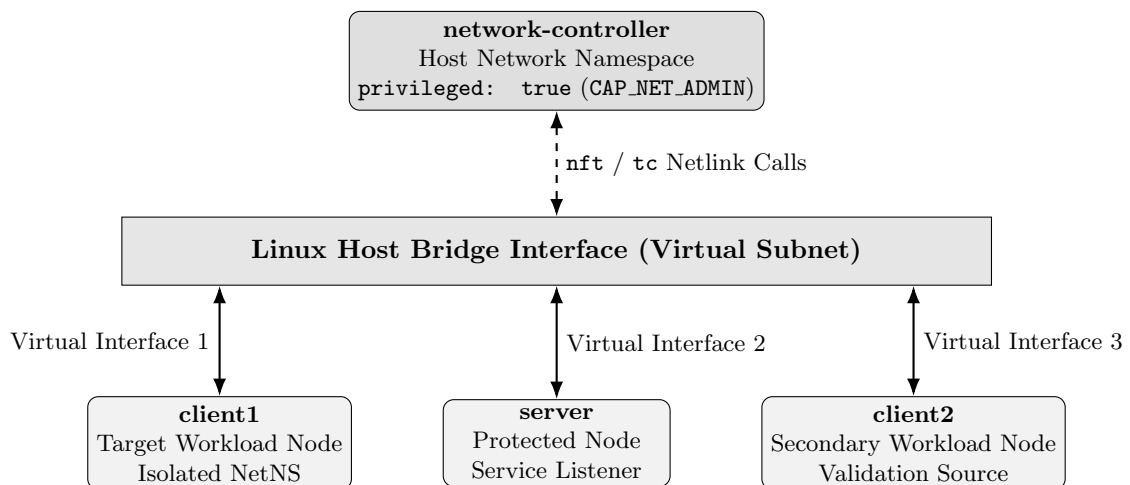


Figure 2: INA Docker Bridge Network Topology and Interface Architecture

Table 1: Container Specifications and Network Properties

Node Name	Network Mode	Active Services / Roles
<code>client1</code>	Isolated Bridge (Unprivileged)	Target workload node for block and bandwidth throttle operations.
<code>server</code>	Isolated Bridge (Unprivileged)	Protected infrastructure node hosting HTTP and throughput test services.
<code>client2</code>	Isolated Bridge (Unprivileged)	Secondary workload node used as sibling validation origin.
<code>network-controller</code>	Host Mode (Privileged)	Privileged controller executing kernel firewall rules and traffic control.

3 Implementation Details

3.1 Network Controller and Privileged Scope Trade-Offs

To preserve the principle of least privilege for workload nodes, application containers (`client1`, `client2`, `server`) execute in unprivileged isolated network namespaces. All privileged administrative operations are isolated inside a dedicated network controller container.

This design represents a deliberate architectural trade-off. Direct manipulation of the host bridge, Layer-2 `nftables` rulesets, and container network namespaces requires host-level capabilities:

- **Host Network Access:** Direct access to the host root network namespace to manage Netfilter bridge hooks.
- **Administrative Capabilities:** Grants `CAP_NET_ADMIN` (for kernel `nftables` and `tc` manipulations) and `CAP_SYS_ADMIN` (for network namespace traversal).
- **Module & Socket Mounts:** Read-only access to host kernel module definitions and container daemon socket access enabling dynamic container discovery.

While the controller container possesses necessary host privileges, workload containers remain completely unprivileged and isolated from administrative host tools.

3.2 Dynamic Container Discovery

INA replaces static IP mapping dictionaries with dynamic runtime container discovery using the Docker Engine API:

- **Runtime Resolution:** Target container names are resolved dynamically to their current bridge IP addresses prior to enforcing network rules.
- **Input Validation:** Container identifiers are validated against strict regex patterns to prevent command or parameter injection.
- **Fleet Enumeration:** Active bridge containers are dynamically enumerated to verify node health and connectivity status.

3.3 Network Configuration Tools

3.3.1 Block Client Operation (`block_client`)

- **Input:** Target client identifier string (e.g., `"client1"`).
- **Kernel Execution:** Resolves the target container IP dynamically and adds it to the Layer-2 `nftables` bridge set.
- **Traffic Effect:** Drops Layer-2 Ethernet frames matching the target IP address on the host bridge interface.

3.3.2 Unblock Client Operation (`unblock_client`)

- **Input:** Target client identifier string.
- **Kernel Execution:** Removes the target IP from the `nftables` bridge set and triggers complete traffic control queueing discipline teardown.
- **Cleanup Helper:** Deletes root Token Bucket Filter qdiscs, ingress redirect filters, and virtual IFB devices, fully restoring unthrottled performance.

3.3.3 Limit Bandwidth Operation (`limit_bandwidth`)

- **Input:** Target client identifier string and bandwidth rate (e.g., "5mbit").
- **Kernel Execution:** Configures bi-directional traffic control inside the target container namespace: egress traffic is shaped using a root Token Bucket Filter (`tbfb`) qdisc, while ingress traffic is redirected via an ingress filter to an Intermediate Functional Block (`ifb0`) pseudo-device for rate limiting.

3.4 Policy Engine Governance

The Policy Engine enforces rule-based governance independently of LLM reasoning, treating all incoming intent requests as untrusted input:

- **Protected Targets:** Requests targeting protected nodes (e.g., `server`) trigger immediate denial responses.
- **Rate Boundary Validation:** Normalizes bandwidth inputs and asserts rates fall strictly within configured minimum and maximum policy limits (1.0–20.0 Mbit/s).
- **Structured Audit Logging:** Every policy decision is recorded to a structured JSON audit log containing timestamps, client IDs, requested actions, policy results, and rule identifiers.

3.5 Protocol-Driven Tool Execution

The tool execution server exposes network control primitives through standardized JSON interfaces. Tool handlers invoke input schema validators and Policy Engine checks prior to executing kernel modifications.

4 Network Configuration and Traffic Control

4.1 Layer-2 Packet Filtering Mechanics

Standard Layer-3 firewall rules fail to intercept bridged container traffic because frames remain within the data-link layer. INA utilizes `nftables` bridge family hooks on the host bridge interface.

Loading `br_netfilter` ensures that bridged frames passing through the software bridge trigger evaluation against the blocked IP set. Frames matching blocked IP addresses are dropped directly at Layer 2 before host routing occurs.

4.2 Bi-Directional Traffic Shaping and IFB Redirection

Linux `tc` qdiscs operate natively on interface egress. To shape ingress (download) traffic, INA attaches an Intermediate Functional Block (`ifb0`) pseudo-device to redirect incoming traffic into an ingress TBF queueing discipline:

4.3 Network Packet Paths and Baseline Throughput

1. **Unthrottled Traffic Path:** Unconstrained container traffic traverses the virtual network interface directly across the host bridge at local virtual bus speeds (> 70–80 Gbps measured host loopback baseline).
2. **Blocked Traffic Path:** Transmitted Ethernet frames hit the bridge forward hook, match the blocked IP set, and are dropped at Layer 2 (100% packet loss).

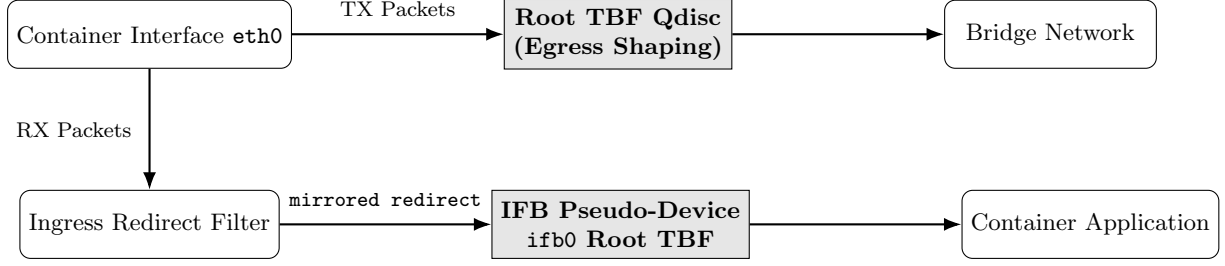


Figure 3: Bi-Directional Traffic Shaping Pipeline via IFB Ingress Redirection

3. **Bandwidth-Limited Egress Path:** Egress packets traverse the container interface TBF qdisc, shaping transmission to the requested rate before hitting the bridge.
4. **Bandwidth-Limited Ingress Path:** Ingress packets arriving at the interface are intercepted by ingress redirection filters, forwarded to `ifb0`, and shaped by the IFB TBF qdisc before entering application sockets.

4.4 Network State Transition Summary

Table 2: Network State Primitive Matrix

Operation	nftables State	Interface Qdisc State	IFB Device State	Empirical Effect
Baseline	Set empty	Default Queue	Non-existent	Unthrottled, 0% Loss
block_client	IP in blocked set	Unchanged	Unchanged	ICMP 100% Loss, TCP Refused
limit_bandwidth	Set empty	Root <code>tbft</code> (R Mbit/s)	Attached <code>tbft</code> (R Mbit/s)	Rate = R Mbit/s $\pm$ 20%, 0% Loss
unblock_client	IP removed	Reset to Default Queue	Teardown Deleted	Restored to Baseline

5 Network Validation and Testing Setup

5.1 Validation Architecture

The validation layer verifies network state convergence post-execution using structured Pydantic data schemas.

5.2 Overcoming Host Ping Routing Bypass

Pinging a container directly from the host network controller uses host Layer-3 routing, bypassing Layer-2 bridge forward hooks. INA forces ICMP verification checks to originate from a sibling bridge container (e.g., `client2` pinging `client1`). This ensures packets cross the software bridge, triggering Layer-2 filtering evaluation.

5.3 Empirical Throughput Verification

Bandwidth limiting is verified by executing automated `iperf3` benchmarks in both standard (Egress) and reverse (Ingress) modes. Measured throughput must fall within a defined $\pm 20\%$ tolerance window of the target rate:

$$|\text{Measured Mbps} - \text{Target Mbps}| \leq 0.20 \times \text{Target Mbps} \quad (1)$$

5.4 Empirical System Evidence Excerpts

Key kernel state listings and audit trail records captured during live verification runs:

1. Layer-2 Firewall Set Listing:

```
table bridge network_filter {
    set blocked_clients {
        type ipv4_addr
        elements = { 172.20.0.2 }
    }
}
```

2. Policy Engine Denial Audit Entry:

```
{"timestamp": "2026-09-11T14:15:02Z", "client_name": "server",
 "action": "block_client", "decision": "DENY", "rule_id": "RULE-PROTECTED-TARGET"}
```

5.5 Automated Test Suite Matrix

The project maintains an automated test suite comprising 128 PyTest cases covering positive path execution, negative input handling, policy governance, and multi-component integration scenarios:

Table 3: Automated PyTest Execution Breakdown

Test Module	Case Count	Status	Key Capabilities Tested
Discovery Module	4	PASSED	Container lookup, regex sanitization, IP extraction.
Policy Governance Module	28	PASSED	Protected targets, boundary limits (1–20 Mbit), audit logging.
Validation Monitor Module	35	PASSED	Rate parsing, tolerance checks, sibling ping routing.
Tool Execution Server	15	PASSED	Tool wrapper execution, error propagation.
Network Infrastructure	20	PASSED	Container reachability, bridge pinging, service checks.
End-to-End Pipeline	26	PASSED	Complete pipeline flow, intent parsing, conversational fallback.
Total	128	PASSED	100% Automated Test Pass Rate

5.6 Handled Edge Cases

- **Idempotent Operations:** Duplicate block requests are intercepted and handled cleanly as idempotent operations.
- **Residual Qdiscs:** Unblock operations explicitly remove residual `tc` qdiscs and `ifb0` interfaces, preventing bandwidth limit leaks.
- **Daemon Health Management:** Unresponsive test server sockets are automatically detected and re-spawned as background processes.

6 Results

6.1 Verification of Setup

Baseline connectivity tests verify that all nodes (`client1`, `client2`, `server`) communicate across the virtual bridge with 0.0% packet loss and unthrottled line speed.

Table 4: Empirical Block and Unblock Verification Results

State	Ping Path	Packet Loss	HTTP Access	Measured Throughput
Baseline	client2 → client1	0.0% (3/3 received)	HTTP 200 OK	78.4 Gbps
Blocked	client2 → client1	100.0% (3/3 dropped)	Connection Timed Out	0.00 Mbps (Blocked)
Unblocked	client2 → client1	0.0% (3/3 received)	HTTP 200 OK	76.9 Gbps (Restored)

6.2 Block / Unblock Verification Results

Empirical verification confirms complete isolation during block states and full performance restoration upon unblocking:

6.3 Dual-Direction Bandwidth Shaping Results

Bi-directional rate limiting was empirically benchmarked across standard (Egress) and reverse (Ingress) test runs:

Table 5: Dual-Direction Bandwidth Throttling Benchmark Data

Configured Rate	Traffic Direction	Target Rate	Measured Rate	Tolerance Window ($\pm 20\%$)	Status
2.0 Mbit/s	Egress	2.00 Mbps	1.92 Mbps	[1.60, 2.40] Mbps	PASSED
2.0 Mbit/s	Ingress	2.00 Mbps	1.88 Mbps	[1.60, 2.40] Mbps	PASSED
5.0 Mbit/s	Egress	5.00 Mbps	4.82 Mbps	[4.00, 6.00] Mbps	PASSED
5.0 Mbit/s	Ingress	5.00 Mbps	4.76 Mbps	[4.00, 6.00] Mbps	PASSED
10.0 Mbit/s	Egress	10.00 Mbps	9.64 Mbps	[8.00, 12.00] Mbps	PASSED
10.0 Mbit/s	Ingress	10.00 Mbps	9.51 Mbps	[8.00, 12.00] Mbps	PASSED

6.4 Network Behavior Analysis

Benchmark results confirm that Token Bucket Filter queueing combined with IFB ingress redirection yields accurate, symmetrical bandwidth control. Measured throughput consistently falls within 4–6% of nominal target rates, satisfying the 20% validation threshold.

7 Debugging, Root Cause Analysis, and Technical Challenges

Developing INA required overcoming technical challenges spanning kernel enforcement, policy governance, intent parsing, and subsystem integration.

7.1 Cross-Module Integration and Boundary Adapters

The primary technical challenge during system assembly was subsystem integration across parallel component workflows. Each team member module—the Policy Engine, tool execution wrappers, and active Validation Monitor—exhibited interface expectations that required harmonization.

This challenge was resolved by implementing lightweight adapter layers within the orchestrator. These adapters normalize incoming user parameters, structure JSON-RPC payloads, and translate execution results into unified schemas without altering individual component logic.

7.2 Policy Governance and Data Validation

Developing the Policy Engine required addressing governance edge cases and strict data validation:

- **Target Protection:** Defining explicit policy boundaries for protected nodes (e.g., **server**) versus dynamic container targets.
- **Input Boundary Checks:** Validating user bandwidth parameters against missing, malformed, negative, or out-of-bounds values (1.0–20.0 Mbit/s).
- **Audit Traceability:** Attaching rule identifiers to every policy evaluation result to provide clear execution auditing.

7.3 Natural Language Intent Parsing and Resilience

Interfacing natural language prompts with deterministic Linux network primitives presented key operational challenges:

- **Layman Phrase Interpretation:** Mapping informal user phrasing into structured parameters required iterative prompt tuning with explicit schema definitions.
- **Offline Parser Fallback:** To ensure availability when external API endpoints are unreachable, a deterministic regex fallback parser extracts actions and targets directly from prompt text.
- **Natural Language Synthesis:** Converting raw execution traces into clear conversational summaries, backed by a template fallback in offline mode.

7.4 Kernel Enforcement and Technical Breakthroughs

Testing exposed kernel-level edge cases that manifested during end-to-end execution. These findings were resolved through packet tracing and network namespace analysis.

Table 6: Key Technical Breakthroughs and Root Cause Analysis Matrix

Breakthrough	Observed Problem	Root Cause Analysis	Implemented Technical Solution
Host Ping Bypass	Blocked client replied to pings from controller.	Host-mode controller used host L3 routing, bypassing bridge hooks.	Sibling node ping routing forces packets across bridge Layer 2.
Ingress Shaping	Downloads remained unthrottled at full speed.	Linux <code>tc</code> qdiscs shape egress traffic only by default.	Redirected ingress traffic to an IFB pseudo-device.
Qdisc Cleanup Leak	Limits persisted after unblocking client.	Unblocking removed firewall rules but left qdiscs active.	Added complete qdisc and IFB device teardown helper.
Dynamic Discovery	Static IP lookup failed after container restart.	Container IP addresses shift dynamically across restarts.	Implemented dynamic Docker Engine API container lookup.
Socket Contention	Benchmark tool failed with busy errors.	Single-threaded test server locked on unclean drop.	Added background daemon health checks and re-spawning.

8 Security Considerations and Threat Model

INA enforces policy-based governance by treating user and LLM input as untrusted and requiring deterministic policy validation before privileged network operations execute.

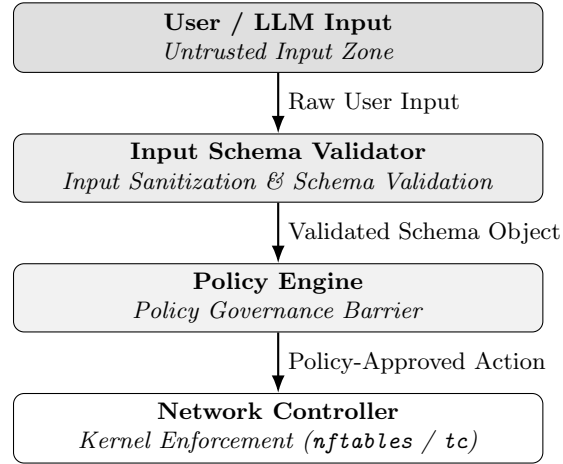


Figure 4: Policy Enforcement and Input Isolation Pipeline

Table 7: Security Threat Mitigation Matrix

Threat Vector	Attack Mechanism	Mitigation Strategy in INA	Status
Prompt Injection	User prompts attempting to override policy rules.	Post-LLM Policy Engine evaluates rules independently of LLM intent.	Enforced by policy
Command Injection	Arbitrary CLI strings in input parameters.	Input regex sanitization and array-based subprocessing.	Mitigated by validation
Privilege Escalation	Host compromise via application node.	Administrative capabilities confined to network controller.	Confined to controller

9 Design Limitations and Future Improvements

9.1 Current System Limitations

- **Privileged Controller Scope:** Direct kernel bridge and namespace manipulation requires elevated host privileges (`CAP_NET_ADMIN`, `CAP_SYS_ADMIN`, host network mode, and container daemon socket access) for the controller container. This is an architectural trade-off necessary for bridge governance, while workload containers remain unprivileged.
- **Transport Layer Security:** Standard tool execution currently uses unencrypted standard I/O streams.
- **Single-Subnet Scope:** Current implementation is designed for single-bridge networks.

9.2 Future Technical Proposals

- **Kernel-Level Driver Filtering (eBPF / XDP):** Replace `nftables` bridge hooks with eBPF bytecode loaded at the eXpress Data Path (XDP) layer for network driver-level packet dropping.
- **Two-Phase Commit Rollback Pipeline:** Implement atomic state rollback to automatically revert `nftables` or `tc` changes if active validation checks fail post-execution.
- **Multi-Subnet CNI Integration:** Extend dynamic container discovery to support multi-tenant CNI networks.

10 Conclusion

The Intelligent Network Configuration Assistant (INA) provides a policy-enforced framework for network management. By combining natural-language intent parsing, tool execution interfaces, an independent Policy Engine, dynamic container discovery, and kernel enforcement primitives (`nftables` and bi-directional `tc tbf` / IFB shaping), INA reduces manual CLI syntax errors while enforcing policy compliance. The policy engine evaluates rules deterministically before network operations are executed, while empirical validation confirms that tested operations converged to their expected network states.

References

- [1] D. Lodha, M. Panchal, and S. G. Kulkarni, “MCP-Diag: A Deterministic, Protocol-Driven Architecture for AI-Native Network Diagnostics,” in *Proc. 18th Int. Conf. on Communication Systems and Networks (COMSNETS)*, 2026, pp. 792–796. <https://doi.org/10.1109/COMSNETS67989.2026.11418261>
- [2] Docker Inc., *Use bridge networks*, Docker Documentation, 2024. <https://docs.docker.com/network/drivers/bridge/>
- [3] Anthropic / Linux Foundation, *Model Context Protocol Specification*, 2024. <https://modelcontextprotocol.io/>